

Graph-Preconditioned Fixed-Effect Absorption

for Digital Experiments

Alexander Fischer¹ and Kristof Schröder²

Extended abstract for CODE@MIT 2026

Abstract. Analysts of platform experiments absorb fixed effects for users, items, markets, and time periods to remove variation that would otherwise enter the standard error on the treatment coefficient. The Method of Alternating Projections (MAP) performs this absorption by demeaning one factor at a time, and it converges slowly when the co-occurrence graph linking the absorbed factors is sparse or close to disconnected. Sparse two-sided exposure in marketplaces and cluster randomization that partitions an interaction graph both produce such graphs. Our preconditioner uses overlapping factor-pair subproblems. Each subproblem is a signed graph Laplacian that admits sparse approximate Cholesky factorization; the resulting operator enters a Krylov solver for least squares. On a near-nested ten-million-observation design, the preconditioned solver completes in 4.63s against 62.2s for the fastest MAP implementation. In the one-million-observation Poisson benchmark, within takes 5.52s and fixest 439.3s. On dense, well-connected graphs MAP remains faster, and the worker-item Gap λ_2 is larger. Because the preconditioner depends on the fixed effects and the weights but not on the right-hand side, one factorization serves the many residualizations that randomization inference and multi-metric readouts require: measured cost per right-hand side falls from 0.137s to 0.047s between one and twenty-five columns.

Keywords: digital experiments; high-dimensional fixed effects; alternating projections; preconditioning; variance reduction

1. Fixed effects as a variance-reduction device

A single experiment readout may record outcomes for users across items, markets, treatment cohorts, and calendar periods. Absorbing these dimensions removes persistent differences in composition and exposure, and it serves the same purpose as the pre-experiment covariate adjustment of (Deng et al. 2013), with the difference that the adjustment is non-parametric in the absorbed dimensions. The resulting regression has a handful of coefficients of interest and millions of nuisance fixed-effect coefficients. How long it takes to remove those nuisance coefficients becomes part of the experiment’s operating cost, and in the hardest cases it decides whether a specification is estimated at all.

The Frisch-Waugh-Lovell theorem reduces the problem to residualizing the outcome and the covariates against the fixed effects before estimating the coefficients of interest (Frisch and Waugh 1933; Lovell 1963). For a right-hand side μ , let D be the sparse fixed-effect design matrix and W a diagonal matrix of observation weights. The residualization problem is

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \| W^{\frac{1}{2}}(\mu - D\alpha) \|_2^2, \quad G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD.$$

The same matrix G serves the outcome and every covariate, so a solver can amortize its setup cost across right-hand sides. We use a preconditioner that draws on the cross-factor structure of G . MAP uses only the diagonal blocks of G and passes cross-factor information through repeated residual updates. We instead solve overlapping pairwise blocks approximately and combine their corrections with an additive Schwarz construction, so that the co-occurrence structure enters the solver directly.

2. Where hard co-occurrence graphs arise in experiments

Absorption cost depends on the shape of the co-occurrence graph rather than on the number of observations alone. Two settings common in platform experimentation generate the sparse and near-disconnected graphs on which MAP is slow, and a third makes the setup cost of any reusable factorization negligible.

2.1. Two-sided exposure in marketplace experiments

A marketplace experiment records outcomes at the level of a pair: a traveller and a hotel, a rider and a driver, a buyer and a listing.

Absorbing user and item effects removes persistent differences in propensity to convert and in item quality. The cross-tabulation C_{UI} counts how often each user is observed with each item, and it is the weighted adjacency matrix of a bipartite graph. The identical object appears in matched employer-employee data as the worker-firm cross-tabulation, which is why the benchmarks below use designs from that literature: they are public, standard, and parameterized by the graph feature we care about.

The two settings differ in density. A worker in an administrative panel is observed at a small number of firms over a career, whereas a user on a platform interacts with a handful of items drawn from a catalogue of millions, and a readout window of two weeks leaves most users observed once. Users who appear with several items act as movers, and they are the only observations that connect item effects to one another. When such users are scarce, user and item effects are nearly confounded within small subgraphs, and each MAP pass carries little information across the graph.

¹trivago

²appliedAI Institute for Europe gGmbH

2.2. Cluster randomization under interference

Where outcomes spill over between units, experimenters partition the interaction graph into clusters and randomize at the cluster level (Ugander et al. 2013; Eckles et al. 2017). The design problem is to choose a partition that cuts as little edge weight as possible, since edges crossing cluster boundaries carry the contamination that biases the estimate.

Both the partition criterion and the links that slow MAP come from the same weighted graph. A partition that cuts little edge weight leaves the corresponding factor-pair block close to block diagonal, with thin bridges between the blocks. The Gap λ_2 defined in Section 3 is then small. A cluster design that limits spillovers can therefore also leave MAP with a graph on which it converges slowly. We report this connection as a structural one; we have not benchmarked a cluster-randomized design directly, and the strength of the effect will depend on how the analysis panel is constructed.

2.3. Repeated residualizations against one fixed-effect structure

Reading out an experiment seldom requires a single residualization. A readout covers several outcome metrics, heterogeneity analysis adds treatment-by-segment interactions, and inference under clustered or switchback assignment often proceeds by randomization, in which the assignment vector is redrawn many times and the test statistic recomputed on each draw (Bojinov et al. 2023). Across all of these right-hand sides the design matrix D and the weights W are unchanged; only the vector being residualized differs. The factor-pair preconditioner depends on D and W alone, so one factorization serves every column, and Section 5 reports the measured decline in cost per right-hand side.

3. Why MAP slows down, and the Gap

Let the factors be user, item, and period. The diagonal blocks of G hold weighted level counts and are themselves diagonal, while the off-diagonal blocks are the cross-tabulations C_{UI} , C_{UP} , and C_{IP} . MAP, also called iterative demeaning or the zig-zag algorithm, updates one factor at a time (Guimaraes and Portugal 2010; Gaure 2013): it computes the weighted mean of the current partial residual within each level of factor q and subtracts it. A complete pass is cheap because every update divides by a diagonal count, and the cross-tabulations are never solved as coupled systems. When the graph is poorly connected, each pass transfers only a small amount of information across the narrow bridges, and many passes are required. Following (Jochmans and Weidner 2019), we call the second-smallest eigenvalue of the normalized Laplacian the Gap. With degree matrix Δ_{qr} , $S_{qr} = \Delta_{qr}^{-\frac{1}{2}} L_{qr} \Delta_{qr}^{-\frac{1}{2}}$ and $\text{Gap}_{qr} = \lambda_2(S_{qr})$. Edge weights are co-occurrence counts, and a smaller Gap indicates near nesting or weak cross-exposure. After singleton removal, a disconnected graph is represented by the component containing the most observations. With three factors, the Gap describes one pair and does not bound convergence of the full model, so we use it as a diagnostic rather than a solver-selection rule.

4. The factor-pair preconditioner

For a factor pair (q, r) , changing the sign of one factor in the pair block gives

$$L_{qr} = \begin{pmatrix} G_{qq} & -C_{qr} \\ -C_{(qr)'} & G_{rr} \end{pmatrix},$$

a weighted bipartite graph Laplacian whose diagonal entries are weighted degrees and whose off-diagonal entries are non-positive edge weights. We solve the pair problem in this representation and reverse the sign afterward. The preconditioner uses one local problem per connected component of every factor pair. If R_{qr} restricts a global coefficient vector to a pair subdomain and Ω_{qr} holds symmetric partition-of-unity weights, the additive Schwarz operator is

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \Omega_{qr} A_{qr} \Omega_{qr} R_{qr},$$

where A_{qr} approximates the inverse of the pair block. With three factors the user-item, user-period, and item-period corrections overlap, and a level appearing in two subdomains receives weight $1/\sqrt{2}$ on each side so that the squared weights sum to one.

Large pair systems cannot be inverted densely. Eliminating one side of a bipartite graph is a division by level counts, but the Schur complement creates clique fill among the neighbours of each eliminated level. We replace these cliques with sparse randomized trees and factor the reduced Laplacian with approximate Cholesky methods (Gao et al. 2025), producing a sparse approximation to each pair inverse. LSMR applies these approximations together with the design operator and its transpose, without forming G (Fong and Saunders 2011). The outer iteration corrects the remaining approximation error, so the algorithm returns the solution of the original problem rather than of a modified objective.

5. Benchmark evidence

We benchmark the full regression path, including factor representation, residualization, low-dimensional estimation, and preconditioner setup. Both designs use ten million observations, one covariate, and three fixed effects; the simple design has dense random exposure and the difficult design a sparse, nearly nested two-sided structure (Bergé et al. 2026). Times are medians over three calls on an Apple M4 Mac mini with 10 cores and 16 GB of memory.

Design	Gap λ_2 (share)	PyFixest MAP	fixest	FEM.jl	within
<i>OLS, 10M observations</i>					
simple (well-connected)	0.622 (1.00)	2.35s	2.52s	2.08s	11.2s
difficult (near-nested)	8.35×10^{-8} (1.00)	326.2s	62.2s	27.0s	4.63s
<i>Poisson, 1M observations</i>					
simple (well-connected)	0.622 (1.00)	7.95s	4.63s	5.71s	9.74s
difficult (near-nested)	8.35×10^{-6} (1.00)	capped	439.3s	129.8s	5.52s

On the simple design the graph is dense, MAP converges in few passes, and within is slowest because the preconditioner does not repay its setup cost. The ranking reverses on the difficult design, where the Gap falls to 8.35×10^{-8} : unaccelerated MAP takes 326.2s and the fastest MAP backend 62.2s, while within completes in 4.63s.

The Poisson rows matter for experiment readouts because platform outcomes are frequently counts of bookings, clicks, or sessions. Iteratively reweighted least squares repeats the demeaning step at every iteration (Correia et al. 2020), so any change in absorption cost is multiplied by the number of iterations. On the difficult design rust-map does not converge within its cap and fixest takes 439.3s, against 5.52s for within. A controlled comparison that varies two-sided mobility while holding the rest of the data-generating process fixed reproduces the pattern: within stays between 0.373s and 0.554s across the designs, whereas MAP reaches its 10,000-pass cap at the lowest mobility. Across a broader set of public benchmark datasets, accelerated MAP wins on small or compact graphs, and the factor-pair preconditioner wins on the larger networks whose hard components cover much of the sample.

6. Amortization across right-hand sides

Randomization inference and multi-metric readouts residualize many vectors against one fixed-effect structure. We measure this directly on a one-million-observation difficult design, comparing the factor-pair preconditioner against diagonally preconditioned LSMR as the number of right-hand sides K grows.

K	diagonal	factor-pair	ratio	factor-pair / RHS
1	3.83s	0.14s	27.9×	0.137s
5	8.09s	0.28s	28.5×	0.057s
25	37.58s	1.16s	32.3×	0.047s

Cost per right-hand side falls by a factor of 2.9 between one and twenty-five columns and is close to flat thereafter, because construction is paid once. A randomization-inference loop with several hundred draws sits far along this curve, so the setup cost that makes the preconditioner unattractive for a single fit on a dense graph is divided by the number of draws. This suggests that the crossover point moves in favour of the preconditioned solver as K grows. We have not benchmarked a full randomization-inference loop against MAP end to end, and we present the extrapolation as an implication of the measured split rather than as a measured result.

In a 100,000-observation numerical check the largest slope difference from MAP is 2.8×10^{-7} ; comparisons use fitted values and residuals, because fixed-effect coefficients themselves depend on normalization. The preconditioned solver uses more memory, with an incremental peak-RSS cost of 178-242 MiB at one million observations.

7. Practical guidance and software

Accelerated MAP remains a good default for dense, well-connected graphs and for one-off fits where setup cannot amortize. The factor-pair preconditioner is intended for sparse two-sided exposure, cluster designs with thin bridges, count outcomes estimated by IRLS, and workloads that residualize many right-hand sides against one fixed-effect structure. A small Gap identifies weakly connected pair graphs; pilot timings should decide whether setup is recovered.

The method is available in the open-source within project (within Developers 2026). The computational core is written in Rust and exposed through Rust (within), Python (within-py), and R (withinr) interfaces, and it is available as a PyFixest demeaning backend (PyFixest Developers 2026). Python users residualize an outcome and several covariates jointly with `solve_batch`, which reuses one preconditioner across the columns. The repository records benchmark inputs, timing boundaries, convergence flags, and external normal-equation residuals, so that a capped MAP run is distinguishable from a successful fit.

References

- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. “Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.”
- Bojinov, Iavor, David Simchi-Levi, and Jinglong Zhao. 2023. “Design and Analysis of Switchback Experiments.” *Management Science* 69 (7): 3759–77.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. “Fast Poisson Estimation with High-Dimensional Fixed Effects.” *The Stata Journal* 20 (1): 95–115.
- Deng, Alex, Ya Xu, Ron Kohavi, and Toby Walker. 2013. “Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data.” In “Proceedings of the Sixth ACM International Conference on Web Search and Data Mining,” Special issue, *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*, 123–32.
- Eckles, Dean, Brian Karrer, and Johan Ugander. 2017. “Design and Analysis of Experiments in Networks: Reducing Bias from Interference.” *Journal of Causal Inference* 5 (1): 20150021.
- Fong, David Chin-Lung, and Michael Saunders. 2011. “LSMR: An Iterative Algorithm for Sparse Least-Squares Problems.” *SIAM Journal on Scientific Computing* 33 (5): 2950–71.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. “Partial Time Regressions as Compared with Individual Trends.” *Econometrica* 1 (4): 387–401.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. “AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization.” *SIAM Journal on Scientific Computing*.
- Gaure, Simen. 2013. “OLS with Multiple High Dimensional Category Variables.” *Computational Statistics & Data Analysis* 66: 8–18.
- Guimaraes, Paulo, and Pedro Portugal. 2010. “A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects.” *The Stata Journal* 10 (4): 628–49.
- Jochmans, Koen, and Martin Weidner. 2019. “Fixed-Effect Regressions on Network Data.” *Econometrica* 87 (5): 1543–60.

- Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010.
- PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python."
- Ugander, Johan, Brian Karrer, Lars Backstrom, and Jon Kleinberg. 2013. "Graph Cluster Randomization: Network Exposure to Multiple Universes." In "Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining," Special issue, *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 329–37.
- within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data."